



YZM0101

Yapay Zeka ve Makine Öğrenimine

Giriş

Hafta 2

Problem Çözmeye Giriş

Yapay Zekanın Temel Görevi

- Geçen hafta: Akıllı ajanlar ve çevreleri. (PEAS, Ajanlar, Çevre, Yapay Zeka Tarihi)
- Bu hafta: Ajanların 'düşünme' süreci nasıl işler?
- Problem çözme, hedefe ulaşmak için bir eylem dizisi bulma sürecidir.
- Birçok YZ problemi, özünde bir 'arama problemi' olarak formüle edilebilir.

İyi Tanımlanmış Problemler

Bir Problemi Formüle Etmek

- Bir problemi çözmeden önce onu net bir şekilde tanımlamalıyız.
- İyi tanımlanmış bir problem dört temel bileşenden oluşur:
- 1. **Başlangıç Durumu (Initial State):** Ajanın başladığı durum.
- 2. **Aksiyonlar (Actions) veya Operatörler:** Ajanın herhangi bir durumda yapabileceği olası eylemler.
- 3. **Hedef Testi (Goal Test):** Mevcut durumun bir hedef durumu olup olmadığını belirler.
- 4. **Yol Maliyeti (Path Cost):** Başlangıçtan hedefe giden bir yolun maliyetini (örn: mesafe, zaman) hesaplar.

Örnek Problem: Romanya Gezgini

- Problem Formülasyonu
 - **Problem:** Arad şehrinden Bükreş'e en kısa yoldan gitmek.
 - **Başlangıç Durumu:** 'Arad' şehri.
 - **Aksiyonlar:** Mevcut şehirden gidilebilecek komşu şehirler.
 - **Hedef Testi:** Mevcut şehir 'Bükreş' mi?
 - **Yol Maliyeti:** İki şehir arasındaki mesafelerin toplamı.



Durum Uzayı (State Space)

Problemin Haritası

- Bir problemdeki erişilebilir tüm durumların kümesidir.
- Durumlar (nodes) ve aksiyonlar (edges) bir graf oluşturur.
- Başlangıç durumundan başlayarak aksiyonlar aracılığıyla bu graf üzerinde bir yol (path) ararız.
- Çözüm, başlangıç durumunu bir hedef durumuna bağlayan bir yoldur.

Arama Ağacı (Search Tree)

Durum Uzayını Keşfetme Yöntemi

- Arama algoritmaları, durum uzayını keşfederken bir 'arama ağacı' oluşturur.
- Ağacın kökü (root), başlangıç durumudur.
- Bir düğümü 'genişletmek' (expanding), o durumdan yapılabilecek tüm aksiyonları uygulamak ve sonuçta ulaşılan yeni durumları (çocuk düğümleri) ağaca eklemektir.
- Arama stratejisi, hangi düğümün bir sonraki adımda genişletileceğini belirler.

Düğüm (Node) Veri Yapısı

Arama Ağacının Yapı Taşı

- Arama ağacındaki her düğüm, aşağıdaki bilgileri içeren bir veri yapısıdır:
- **STATE**: Düğümün temsil ettiği durum.
- **PARENT**: Bu düğüme hangi düğümden gelindiği (ata düğüm).
- **ACTION**: Ata düğümden bu duruma gelmek için uygulanan aksiyon.
- **PATH-COST**: Kökten bu düğüme kadar olan toplam yol maliyeti (genellikle ' $g(n)$ ' olarak gösterilir).

Arama Algoritmalarını Değerlendirme Kriterleri

Bir Arama Stratejisi Ne Kadar İyidir?

- **Tamlık (Completeness):** Eğer bir çözüm varsa, algoritma onu bulmayı garanti ediyor mu?
- **Optimalite (Optimality):** Algoritma, en düşük yol maliyetine sahip çözümü (optimal çözümü) bulmayı garanti ediyor mu?
- **Zaman Karmaşıklığı (Time Complexity):** Çözümü bulmak ne kadar sürer? (Genişletilen düğüm sayısı cinsinden ölçülür).
- **Uzay (Yer) Karmaşıklığı (Space Complexity):** Arama sırasında hafızada ne kadar düğüm tutulması gerekir?

Karmaşıklık Ölçütleri

b, d, m Değişkenleri

- Arama algoritmalarının karmaşıklığını ifade etmek için genellikle şu değişkenleri kullanırız:
- **b (branching factor):** Dallanma faktörü. Bir düğümün sahip olduğu ortalama çocuk sayısı.
- **d (depth):** Derinlik. En sık (en az maliyetli) hedef durumunun köke olan uzaklığı.
- **m (maximum depth):** Durum uzayının maksimum derinliği (sonsuz olabilir).

Arama Stratejilerine Genel Bakış

Bilgisiz(Uninformed) ve Bilgili(Informed) Arama

- **Bilgisiz Arama (Uninformed Search):**

- Sadece problem tanımındaki bilgileri kullanır.
- Hedefe ne kadar yakın veya uzak olduğu hakkında hiçbir fikri yoktur.
- Kör arama (blind search) olarak da adlandırılır.
- Örnekler: BFS, DFS, UCS.

- **Bilgili Arama (Informed Search):**

- Probleme özgü ek bilgileri (sezgiselleri) kullanır.
- Hangi düğümün hedefe daha yakın olduğuna dair bir 'tahminde' bulunur.
- Genellikle çok daha verimlidir.
- Örnekler: Greedy Search, A*.

Tekrarlanan Durumlar ve Döngüler

Arama Sırasındaki Tehlikeler

- Arama sırasında aynı duruma farklı yollardan tekrar ulaşabiliriz (tekrarlanan durum).
- Eğer bir yolu takip ederken daha önce geçtiğimiz bir duruma geri dönersek, bu bir 'döngü' (cycle) oluşturur.
- Döngüler, bazı algoritmaların (örn: standart DFS) sonsuz bir yola sapmasına neden olabilir.
- Çözüm: Daha önce ziyaret edilen durumların bir listesini (closed list veya explored set) tutmak.

Soyutlama (Abstraction)

Problem Çözmede Anahtar Kavram

- Gerçek dünya problemleri son derece karmaşıktır.
- Problem formülasyonu, gerçek dünyadaki detayların soyutlanması işlemidir.
- Gereksiz detayları (örn: arabanın rengi, radyonun çaldığı şarkı) atarak problemin özüne odaklanılır.
- Doğru soyutlama seviyesini bulmak, çözülebilir bir model oluşturmak için kritiktir.

Örnek Problem:

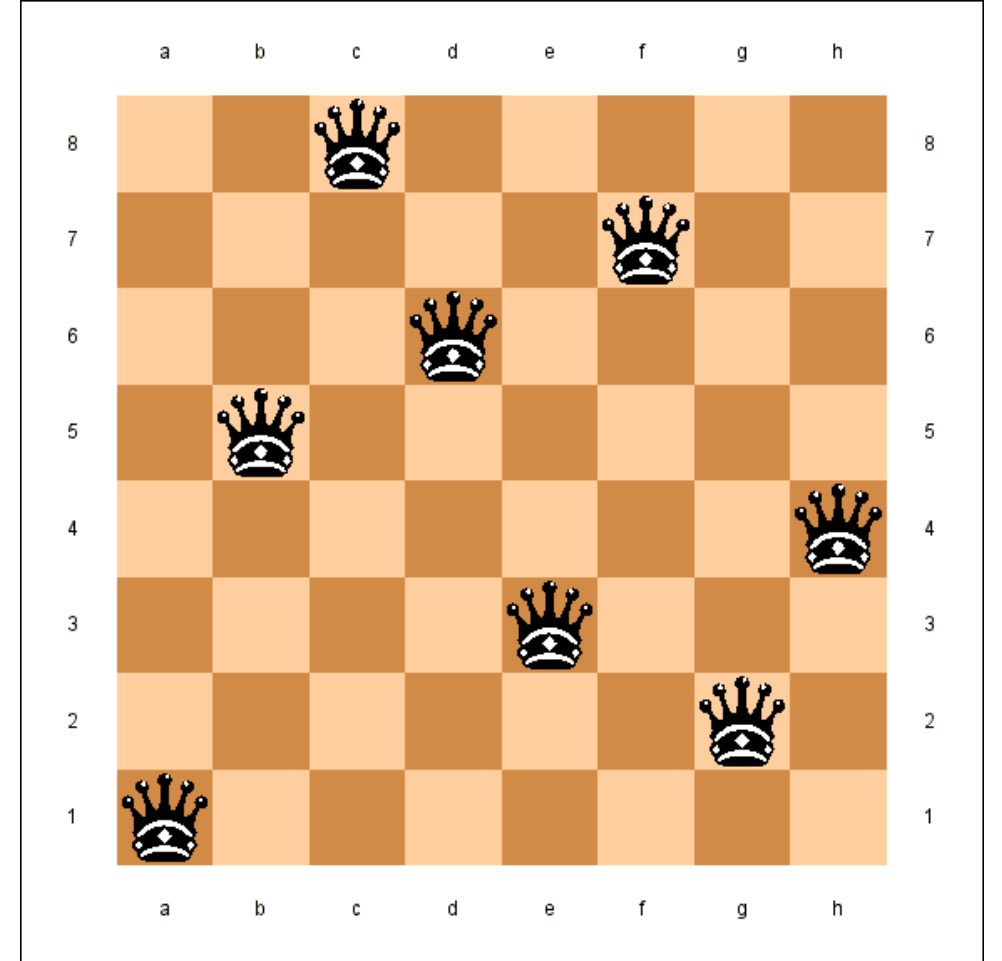
8-Bulmaca

- Klasik Bir Arama Problemi
 - **Durumlar:** 3x3'lük bir tahtadaki 8 karonun ve boşluğun konfigürasyonları.
 - **Başlangıç Durumu:** Karışık bir tahta konfigürasyonu.
 - **Aksiyonlar:** Boşluğu Yukarı, Aşağı, Sola veya Sağa kaydırmak.
 - **Hedef Testi:** Tahtanın, hedef konfigürasyona ulaşıp ulaşmadığı.
 - **Yol Maliyeti:** Her hamle 1 maliyetindedir. Toplam hamle sayısı.



Örnek Problem: 8-Vezir Problemi

- Kısıt Sağlama Problemi
 - **Problem:** 8x8'lik bir satranç tahtasına 8 veziri, birbirlerini tehdit etmeyecek şekilde yerleştirmek.
 - **Durumlar:** Tahtaya k vezirin ($0 \leq k \leq 8$) yerleştirildiği herhangi bir konfigürasyon.
 - **Başlangıç Durumu:** Boş bir satranç tahtası.
 - **Aksiyonlar:** Bir sonraki boş sütuna bir vezir yerleştirmek.
 - **Hedef Testi:** 8 vezir tahtada ve hiçbirini birbirini tehdit etmiyor.
 - **Yol Maliyeti:** Genellikle bu problemde maliyet önemli değildir, sadece çözümün bulunması yeterlidir.



Gerçek Dünya Problemleri

Soyut Problemlerden Uygulamaya

- **Rota Bulma:** Navigasyon sistemleri, tur planlama. (Arama Problemi)
- **Lojistik ve Çizelgeleme:** Kamyon rotalama, üniversite ders programı oluşturma. (Optimizasyon Problemi)
- **Robotik:** Bir robot kolunun bir nesneyi yakalamak için hareket dizisi planlaması. (Yol Planlama)
- **Otomatik Programlama:** Belirli bir işi yapan bir bilgisayar programı tasarlama.

Problem Formülasyonunun Önemi

Doğru Başlamak, Yarı Yarıya Bitirmektir

- Aynı problem, farklı şekillerde formüle edilebilir.
- İyi bir formülasyon, durum uzayını küçültebilir ve çözümü önemli ölçüde hızlandırabilir.
- Durum temsilini nasıl seçtiğiniz, aksiyonların tanımı ve soyutlama seviyesi, performansı doğrudan etkiler.
- Bu, genellikle deneyim ve yaratıcılık gerektiren bir adımdır.

Altyapı: Sınır (Frontier) ve Keşfedilenler (Explored Set)

Arama Algoritmalarının Hafızası

- Arama algoritmaları genellikle iki ana veri yapısı kullanır:
- **Frontier (Sınır):** Henüz genişletilmemiş, ancak keşfedilmiş düğümlerin tutulduğu koleksiyon. Bir sonraki adımda hangi düğümün seçileceği buradan belirlenir.
- Farklı algoritmalar, frontier için farklı veri yapıları kullanır (Kuyruk, Yığın, Öncelikli Kuyruk).
- **Explored Set (Keşfedilenler Kümesi):** Daha önce genişletilmiş olan durumların tutulduğu küme.
- Tekrarlanan durumları ve döngüleri önlemek için kullanılır.

Genel Arama Algoritması (Tree- Search)

Tüm Algoritmaların Temel İskeleti

```
function TREE-SEARCH(problem):  
  frontier = { INITIAL-STATE(problem) }  
  loop do:  
    if EMPTY?(frontier) then return failure  
    node = CHOOSE-NODE(frontier)  
    if GOAL-TEST(problem, STATE(node)) then return SOLUTION(node)  
    frontier = frontier + EXPAND(node, problem)
```


Graf Arama (Graph-Search) Algoritması

Tekrarlanan Durumları Önleyen Versiyon

```
function GRAPH-SEARCH(problem):  
  frontier = { INITIAL-STATE(problem) }  
  explored = {}  
  loop do:  
    if EMPTY?(frontier) then return failure  
    node = CHOOSE-NODE(frontier)  
    if GOAL-TEST(problem, STATE(node)) then return SOLUTION(node)  
    add STATE(node) to explored  
    for each child in EXPAND(node, problem):  
      if STATE(child) is not in explored or frontier:  
        add child to frontier
```

Haftanın Kavramları: Özet

Akılda Kalması Gerekenler

- **Problem Formülasyonu:** Bir problemi Başlangıç Durumu, Aksiyonlar, Hedef Testi ve Yol Maliyeti ile tanımlama.
- **Durum Uzayı:** Bir problemdeki tüm erişilebilir durumların oluşturduğu graf.
- **Arama Ağacı:** Arama sırasında durum uzayının keşfedilen kısmını temsil eden yapı.
- **Değerlendirme Kriterleri:** Tamlık, Optimalite, Zaman ve Uzay Karmaşıklığı.
- **Bilgisiz vs. Bilgili Arama:** Hedefe dair ek bilgi kullanıp kullanmama durumu

Bilgisiz Arama Stratejileri

Teorik Dersin Konusu

- Genişlik Öncelikli Arama (Breadth-First Search - BFS)
- Tekdüze Maliyetli Arama (Uniform-Cost Search - UCS)
- Derinlik Öncelikli Arama (Depth-First Search - DFS)
- Bu algoritmaların çalışma prensipleri, avantajları, dezavantajları ve karmaşıklık analizleri.

Alıştırma: Vakum Temizleyici Robot

- Problem Formülasyonu Pratiğı
 - • ****Ortam:**** Sadece iki odadan (A ve B) oluşan bir dünya.
 - • ****Ajan:**** Vakum temizleyici robot.
 - • ****Sensörler:**** Robot, bulunduğu odanın temiz mi kirli mi olduğunu algılayabilir.
 - • ****Aksiyonlar:**** Sola git, Sağa git, Temizle.
 - • ****GÖREV:**** Bu problemi, iyi tanımlanmış bir problem olarak formüle edin (Başlangıç Durumu, Aksiyonlar, Hedef Testi, Yol Maliyeti).



Önemli Not: Soyutlama Seviyesi

- Problem formülasyonu yaparken, problemin özüne odaklanın.

- Vakum robotu örneğinde, robotun pil seviyesi, tekerleklerinin dönüş hızı veya sensörünün markası gibi detaylar genellikle önemsizdir.

- Önemli olan, durum tanımını ve aksiyonları, hedefe ulaşma mantığını etkileyen minimum bilgiyle yapmaktır.

- Bu, 'hesaplamalı düşünme'nin (computational thinking) temel bir parçasıdır.



- Sorular